

基于图匹配的模拟电路网表标注与版图约束提取

近几十年来,随着现代科技的快速发展,芯片电路规模的也不断增加,不同于数字电路领域自动化工具的高度成熟,模拟电路领域自动化工具仍存在较大困难,特别是在模拟电路版图自动化领域。当前高性能模拟电路的性能易受工艺变化和版图质量影响,模拟电路版图生成布局布线过程需遵守大量约束以保证电路性能,生成高质量的约束条件对版图生成布局布线过程至关重要。电路自动化标注通过子电路识别和模块标注的方法生成电路分层结构,能应用于模拟版图约束提取和版图优化。

一、网表标注与约束提取

1.1 基于图匹配的网表标注与约束提取

电路网表标注问题本质上是图匹配问题,就是将多个子电路从完整电路中划分出来,一般情况下将子电路作为模式图,将电路图称为待标注图,然后通过图搜索匹配方法将模式图全部匹配。基于图匹配结果以及子电路本身已知的电路约束,可以得到待标注电路图的所有版图约束。

图匹配算法分为精确匹配与近似匹配。其中,图精确匹配要求匹配结果与模式图完全一致,近似匹配则允许一些错误与噪声的匹配结果,对于较大规模的电路,近似匹配可以有效提高匹配效率。但为实现最终的电路板图约束提取,则需要通过精确匹配方法获得点与点的精确映射并进一步得到约束的精确映射。

网表标注与约束提取可以大致分为三个过程。首先,将电路网表表示为图形式,一般情况下可以将线网和器件表示为图节点或者边,然后使用编码或者异构图表达线网与器件的连接关系;然后,结合图匹配算法,得到图匹配结果,进行子电路划分;最后基于匹配结果和子电路约束映射文件,生成整个电路的版图约束。

1.2 电路图表示方法

当前模拟电路图表示方法基本可分为三类:1. 器件抽象为图节点,线网抽象为边;2. 器件抽象为边,线网抽象为节点;3. 器件与线网均抽象为节点。还包括一些新提出的表示方法:S3DET^[1]的器件端口节点图,带有信号流向的异构多重图^[3],连接关系编码图^[4,5],结构信号流图^[8]。基于上述图表示方法,再采用图相似计算方法结合图搜索算法或者图神经网络算法结合机器学习解决电路网表标注任务。

本方案算法采用基于第3种电路图(图1)表示方法的改进方法,如图2所示将器件和线网均抽象为节点,边采用三位二进制编码表示线网与器件端口的连接关系,连接关系编码也抽象为图节点。无向三分图表示为 $G(V,E)$,节点集 V 包含三部分 V_e, V_n, V_c ,分别表示器件节点和线网节点以及连接关系编码节点,边集 E 表示节点之间的直接相连关系,同类节点之间没有边连接。电路器件分为晶体管和無源器件类型,PMOS和NMOS型晶体管包含D、G、S三种Pin,每条连接到晶体管的编码节点表示为3位二进制编码 $I_S I_G I_D$ 。其中 $I_G = 1$,当且仅当编码节点与晶体管G端连接,否则 $I_G = 0$;同理 $I_S = 1(I_D = 1)$,当且仅当编码节点与晶体管S(D)端连接,否则 $I_S = 0(I_D = 0)$ 。连接到無源器件的编码节点则采用编码“1000”表示。

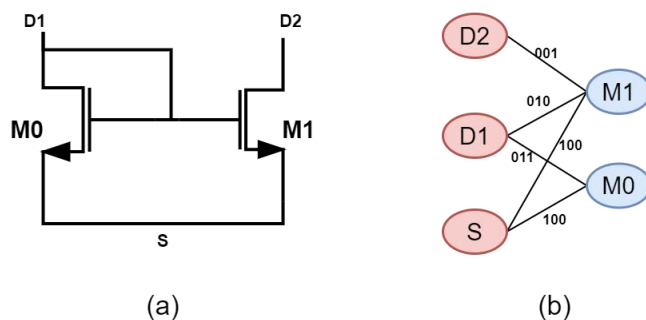


图 1 二分图电路图表示方法 $G(V,E)$ ，节点集 V 包含三部分 V_e, V_n, V_c 分别表示器件和线网，边集 E 表示线网与器件的连接关系（二进制编码）

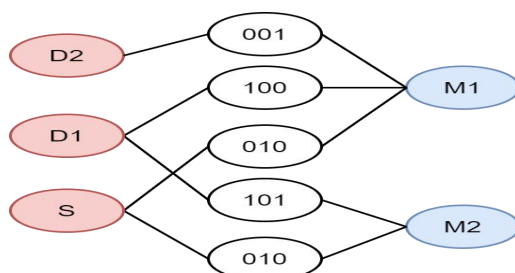


图 2 三分图电路图表示方法 $G(V,E)$ ，节点集 V 包含三部分 V_e, V_n, V_c ，分别表示器件节点和线网节点以及连接关系编码节点（二进制编码），边集 E 表示节点之间的直接相连关系

二、版图约束实例

2.1 REFF_amp1 电路子电路结构

子电路其中包含电流镜、差分对、Load 等微结构，每种微结构对应特定的版图约束。

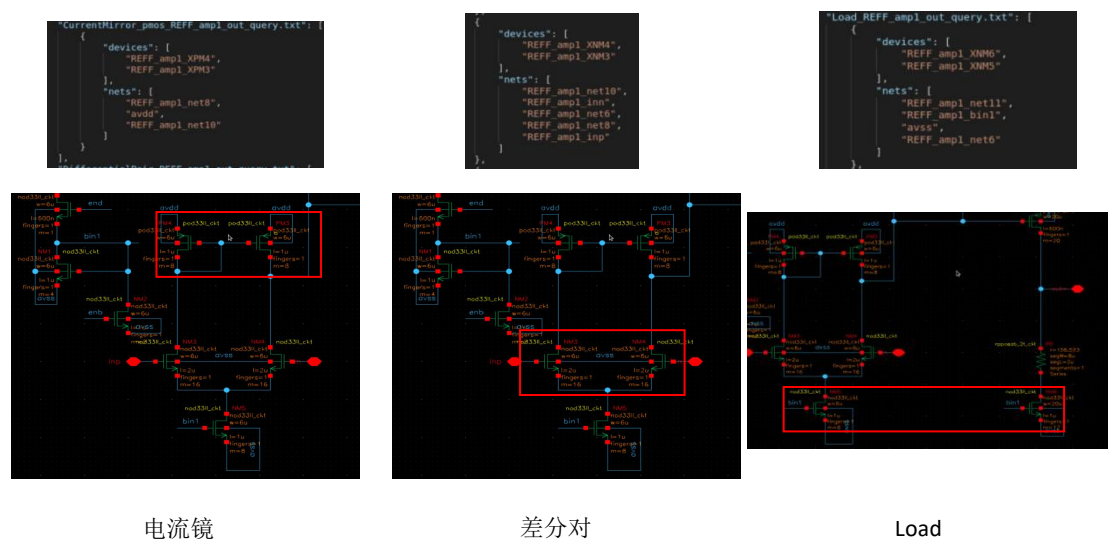


图 3 微结构

2.2 AACCSAR_TOP 电路子电路结构

AACCSAR_TOP 电路有 2000 左右个器件,能够在 10s 内提取完成本项目所需的子电路结构;对于比较简单的子结构,提取效率能达到 115ms/每百个器件。

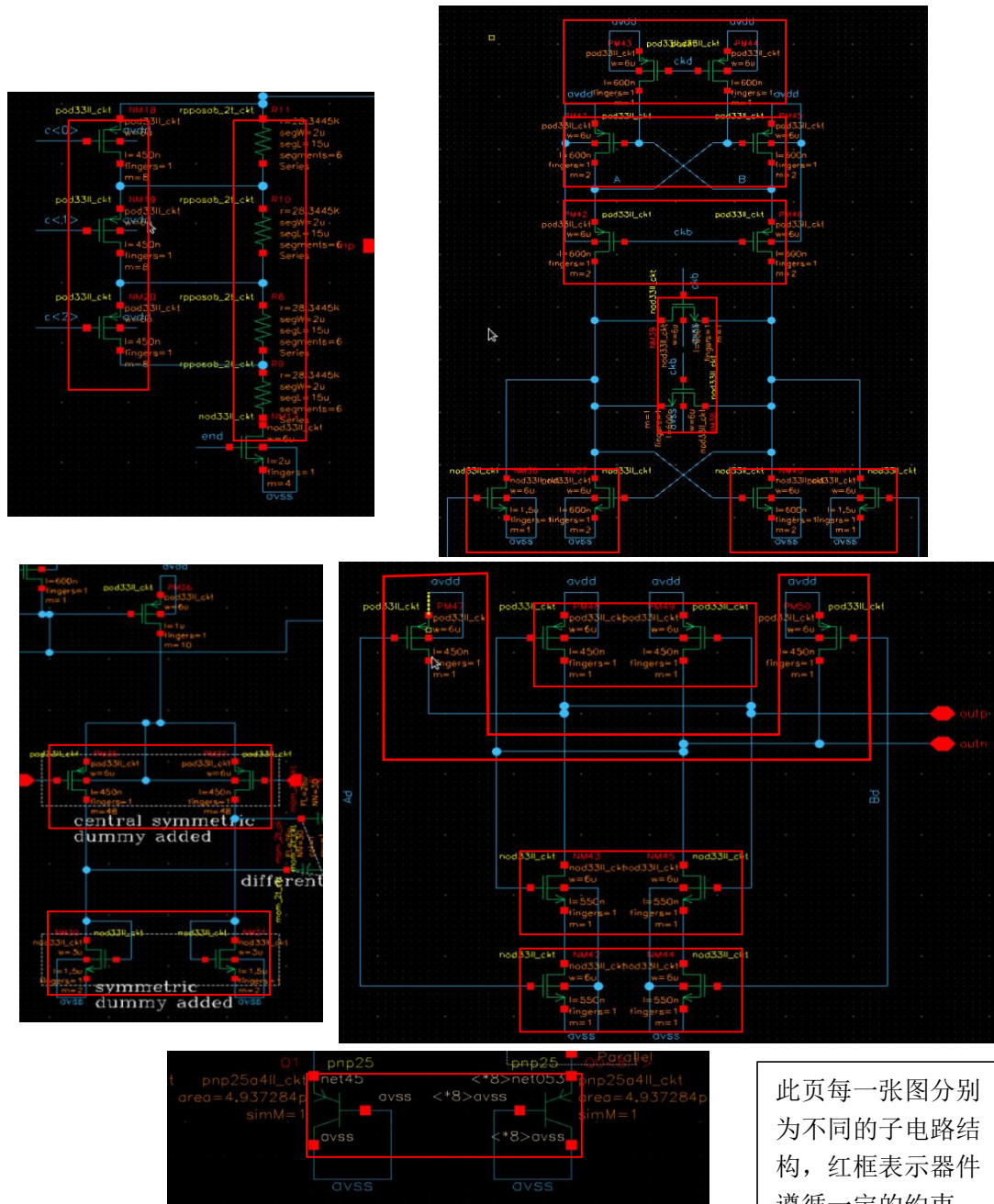


图 4 AACCSAR_TOP 电路约束

三、技术实现

采用图匹配技术，建立子电路模板库，将待标注模拟点电路中包含的所有子电路进行识别，再通过约束映射的方法得到模拟电路的版图约束。

3.1 数据结构

3.1.1 Python 部分

类名	作用
spiceParser	网表解析类
Parameter, Args	输入参数类
CircuitAnnotation	电路标注类，用于调用 C++代码和解析结果文件，并生成最终约束文件
BasicElement	基础组件类，包含电容、电阻、电感、晶体管、PNP 节

3.2 图匹配算法

图匹配算法实现代码来源于一个 Github 开源库：

[RapidsAtHKUST/SubgraphMatching: In-Memory Subgraph Matching: An In-depth Study by Dr. Shixuan Sun and Prof. Qiong Luo \(github.com\)](#)

参考文献：In-Memory Subgraph Matching: An In-depth Study(SIGMOD2020 港科大)

3.2.1 开源库介绍

开源库实现了 8 个代表性内存的性能子图匹配（子图查询）算法。具体来说，将 QuickSI、GraphQL、CFL、CECI、DP-iso、RI 和 VF2++ 实现在一个通用框架中。

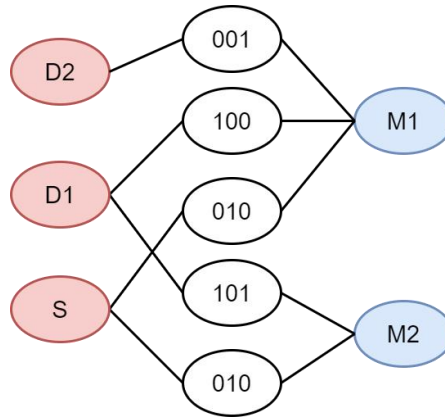
3.2.2 开源库使用

目前已经将开源库集成到 Nimbus 中 ams_apr_adc_subgraph(_main) 模块，可调用 autoGenerateLayoutSubgraphMatch 命令完成图匹配功能。

3.2.2.1 调用命令行格式

```
autoGenerateLayoutSubgraphMatch -d 待标注“.graph”文件 -q 模板“.graph”文件 -filter filter 类型 -order order 类型 -engine engine 类型 -result 结果“.txt”文件路径
```

3.2.2.2 “XX.graph”图文件生成



如上图所示，将电路先表示为二分图，节点分别代表线网和器件，之间的边具有二进制编码，为了使用开源图匹配代码，需要再将边抽象为节点，并将图转化为“.graph”文件格式。

```

1  t 12 12
2  v 0 10 3
3  v 1 0 2
4  v 2 0 2
5  v 3 0 1
6  v 4 9 3
7  v 5 0 1
8  v 6 1 2
9  v 7 4 2
10 v 8 2 2
11 v 9 1 2
12 v 10 4 2
13 v 11 2 2
14 e 0 6
15 e 1 6
16 e 0 7
17 e 2 7
18 e 0 8
19 e 3 8
20 e 1 9
21 e 4 9
22 e 2 10
23 e 4 10
24 e 4 11
25 e 5 11
26

```

如图为 “.graph” 格式，开头为“t”的行后两个数字表示节点数目和边数目；
 开头为“v”的表示节点，后 3 个数字分别表示节点 id，节点类型，节点度；
 开头为“e”的表示边，后两个数字表示边的两端节点 id。

3.2.2.3 result 文件解析

```
result > tmp_REF
1 0 14
2 1 15
3 2 16
4 3 10
5 4 11
6 5 12
7 6 13
8 7 9
9 8 8
10 9 3
11 10 26
12 11 22
13 12 27
14 13 24
15 14 25
16 15 32
17 16 7
18 17 6
19 18 66
20 19 67
21 20 59
22 21 68
23 22 69
24 23 58
25 24 57
26 25 60
27 26 61
28 27 63
```

Result 文件格式如图所示，第一列表示模板文件的节点 id，第二列表示待匹配文件的节点 id，表示节点的匹配映射对应关系。通过解析该文件并根据之前的节点名和 id 映射文件得到匹配结果。

3.3 约束映射

3.3.1 子电路模板文件和约束映射文件

子电路模板文件保存路径：/Nimbus/resource/primitive_spice1/

对应的约束映射文件保存路径：/Nimbus/resource/primitive_constraint_map/

子电路模板文件格式为普通网表格式后缀为“.sp”或“.netlist”，每一个模板文件分别对应一个约束映射文件，命名格式“网表名_map.json”。

3.3.1.1“XX_map.json”文件编写

Key 值	约束类型	Value 格式
1	共质心约束	一个二维数组，每个元素表示不同的遵守“共质心约束”的器件对 group。例如：[['PM0','PM1'], ['NM0','NM1']]表示两个共质心约束器件对 group。
2	对称约束	一个二维数组，每个元素表示不同的遵守“对称约束”的器件对。例如：[['PM0','PM1'], ['NM0','NM1']]表示两个对称约束器件对 group。
3	对称组	一个三维数组，每个元素表示不同的遵守“对称组约束”的对称组，对称组是

	约束	一个二维数组，每个元素表示遵守“对称约束”的器件对。例如： [[['PM0','PM1'],['NM0','NM1']], [['PM2','PM3'],['NM2','NM3']]]表示两个对称组 group，对称组 group 内部的器件对布局在一起，公用一条对称轴。
4	放置约束	一个二维数组，每个元素表示遵守“放置约束”的器件 group，布局时放一起然后作为一个 group。例如：[['PM0','PM1'],['NM0']]表示两个“放置约束”group，每一个 group 内部的器件布局在一起。
5	电容阵列	表示电容阵列遵守的约束
g	group 约束	表示 group 之间应当遵守的约束。group 约束也可包含“1”“2”“3”“4”种约束(目前只使用“3”约束)。其中每一个 group 的表示格式为[key 值,position]表示该 group 为上述“key”约束的第 position 个 group。

3.3.1.2 约束实例

例如: inv_33X1_map.json <pre> { "4":[['PM0'],['NM0']], "g":{ "3":[[['4",0],["4",0]], [['4",1],["4",1]]] } }</pre>
--

解读：这是 inv_33X1.sp 的约束文件。“4”表示“放置约束”，[“PM0”]和[“NM0”]分别遵守该约束：“PM0”单独放置，“NM0”单独放置；

“g”表示 group 遵守的约束，其中“3”表示这些 group 遵守对称组约束，对称组只有一个：[[["4",0],["4",0]], [["4",1],["4",1]]]。

对称组里包含两个 group 对称对。[“4”,0]表示遵守 4 约束的第一个 group，即[“PM0”], [["4",0],["4",0]]表示[“PM0”]这个 group 遵守自对称，[“NM0”]同理，[“PM0”]和[“NM0”]共同构成一个对称组，共用一条对称轴。

3.3.1.3 约束优先级文件“priority.json”

由于各个模板电路遵守的约束在某些情况下会存在冲突,该优先级文件包含了约束映射的优先顺序。越靠前的约束具有更高的优先级,如果某些器件已经遵守某种约束,则对后续的映射约束则自动忽略。

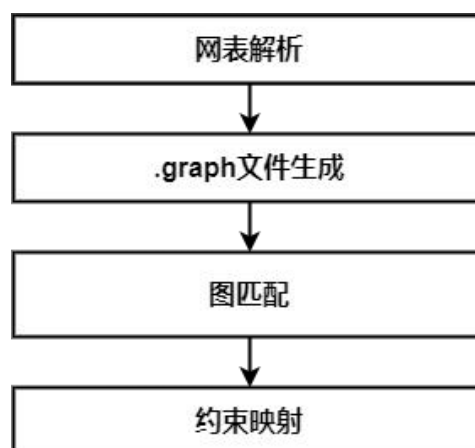
3.3.2 约束映射流程

根据约束优先级文件,按照文件中的顺序分别解析各模板文件包含的约束。并且只保留属于同一子电路的约束。

3.4 顶层逻辑

模拟电路版图约束提取流程可分为三个过程:1)网表解析和文件生成;2)图匹配;3)约束映射。

1. 网表解析和文件生成过程完成网表解析工作,将电路图转化为 `flatten` 过的二分图,再生成待标注电路和模板子电路的‘.graph’文件;
2. 调用图匹配命令行,完成模板匹配。得到‘.txt’结果文件。
3. 约束映射:将模板子电路对应的约束映射到待标注电路中,生成待标注电路各个电路模块遵守的约束。



参考文献

- [1] M. Liu et al., "S3DET: Detecting System Symmetry Constraints for Analog Circuits with Graph Similarity," 2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC), 2020, pp. 193-198, doi: 10.1109/ASP-DAC47756.2020.9045109.
- [2] X. Gao, C. Deng, M. Liu, Z. Zhang, D. Z. Pan and Y. Lin, "Layout Symmetry Annotation for Analog Circuits with Graph Neural Networks," 2021 26th Asia and South Pacific Design Automation Conference (ASP-DAC), 2021, pp. 152-157.
- [3] H. Chen, K. Zhu, M. Liu, X. Tang, N. Sun and D. Z. Pan, "Universal Symmetry Constraint Extraction for Analog and Mixed-Signal Circuits with Graph Neural Networks," 2021 58th ACM/IEEE Design Automation Conference (DAC), 2021, pp. 1243-1248, doi: 10.1109/DAC18074.2021.9586211.
- [4] P. -H. Wu, M. P. -H. Lin and T. -Y. Ho, "Analog layout synthesis with knowledge mining," 2015 European Conference on Circuit Theory and Design (ECCTD), 2015, pp. 1-4, doi: 10.1109/ECCTD.2015.7300111.
- [5] Y. -H. Chen, H. -Y. Chi, L. -Y. Song, C. -N. J. Liu and H. -M. Chen, "A Structure- Based Methodology for Analog Layout Generation," 2019 16th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD), 2019, pp. 33-36, doi: 10.1109/SMACD.2019.8795227.
- [6] K. Kunal et al., "GANA: Graph Convolutional Network Based Automated Netlist Annotation for Analog Circuits," 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2020, pp. 55-60, doi: 10.23919/DATE48585.2020.9116329.
- [7] Patel, Rituj, Husni Habal, and Konda Reddy Venkata. "Machine Learning based Structure Recognition in Analog Schematics for Constraints Generation." *Mirror* 6 (2021): N7.
- [8] Eick, Michael & Strasser, Martin & Lu, Kun & Schlichtmann, Ulf & Graeb, Helmut. (2011). Comprehensive Generation of Hierarchical Placement Rules for Analog Integrated Circuits. *Computer-Aided Design of Integrated Circuits and Systems*, IEEE Transactions on. 30. 180 - 193. 10.1109/TCAD.2010.2097172.